

给 AI 一点 “惦记感”

一种可审计、可插拔、与对话渠道解耦的主动注意力层

外部事实 → 值得看一眼？ → 此刻怎么做？

中文架构解释 + AI 实现索引

2026.08 · github.com/Aryuan026/HermesAttentionCore

先说人话

它补的不是智商，而是“下一次想起什么”

聊天 Agent 与闹钟之间，缺了一层活的判断。

普通聊天 Agent

你叫它，它才动。上下文很鲜活，但没有稳定的后台惦念。

传统 Cron / 提醒器

时间到了就发预写文字。能准时，却不知道此刻是否仍值得打扰、该用什么工具、该说什么。

Attention Core

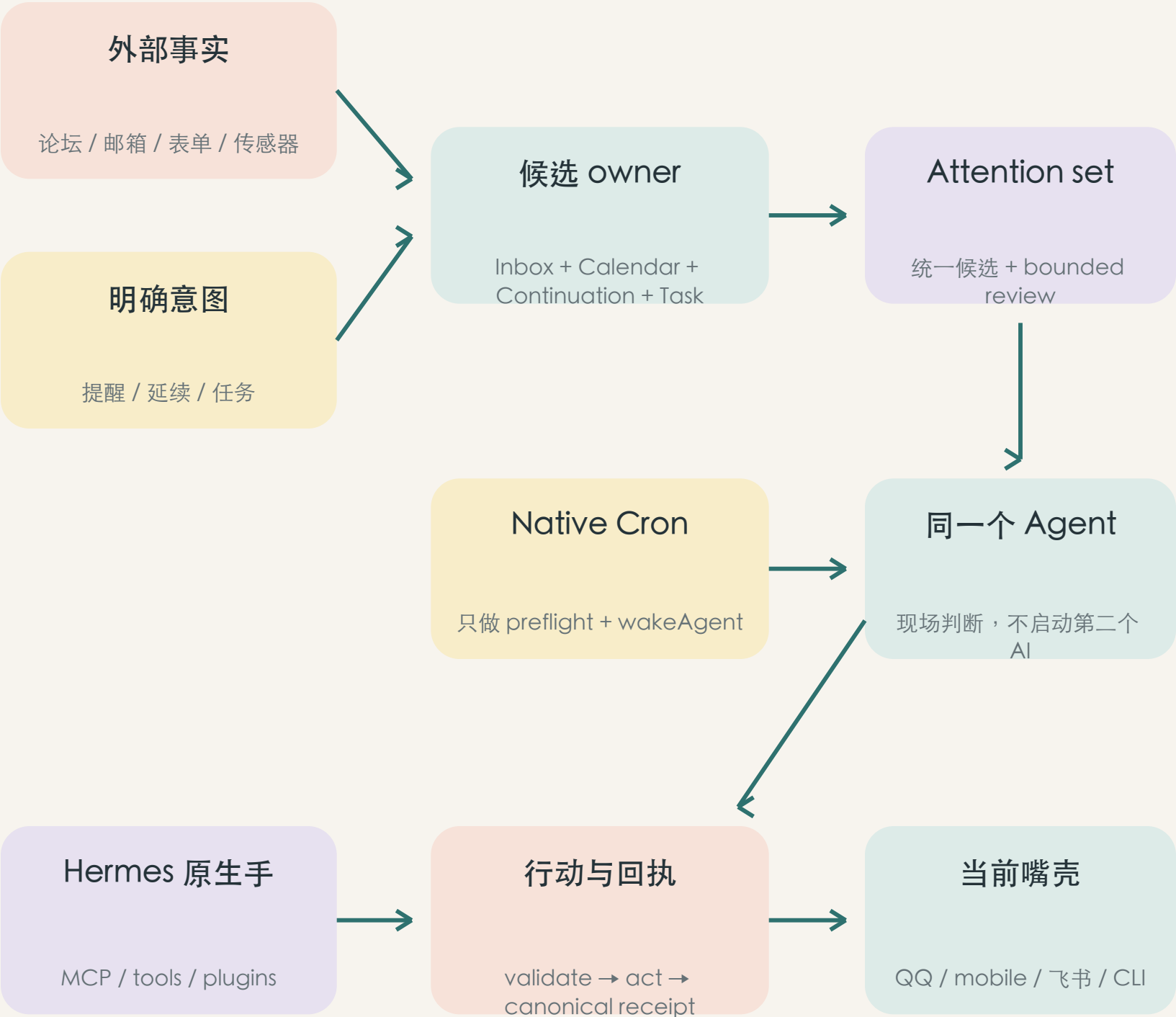
只问：现在有没有什么值得让同一个活的 Agent 看一眼？

提醒文字是上下文，不是最终回复；外部事件是事实，不是行动命令。

全局地图

一条链路，三种归属

刺激进入 Attention；能力留在 Hermes；表达回到当前渠道。



没有 channel-specific Attention path，也没有 Attention 自己的工具注册表。

核心边界

刺激、手、嘴，必须分开

这就是它能换渠道、接新 MCP、接新信息源而不长歪的原因。

刺激 stimulus

01

有什么值得看一眼？

Inbox / Calendar / Continuation / Task

能力 capability

02

我现在有哪些手可用？

Hermes native MCP / tools / plugins

嘴 mouth

03

最后从哪里说话？

QQ / mobile / 飞书 / CLI / future UI

换 QQ 只是换嘴；接邮箱事件是加 Inbox adapter；让 AI 回复邮箱则是另接一个原生能力。

决策路径

不是“消息来了就做”

它更像人：先收拢，再挑一件，确认没变，最后才行动。

维护

1

poll adapter · recover lease · maintain cycles

直达

2

到期 Calendar 走 direct lane

成盘

3

Continuation + Task + Inbox → AOS

收拢

4

34/25/18/11/8/4 权重 + diversity

呈现

5

完整展示 bounded review，默认最多 12 条

选择

6

exact-claim 一条，或 quiet 已看过的一小盘

找手

7

tool_search → describe → call

复核

8

focus validate : lease / version / semantics

收尾

9

canonical receipt → 新鲜表达或沉默

provider priority 只有 4%：外部新消息不会天然抢走全部注意力。

最关键的实现

三个身份，不能混成一个 ID

事实、集合、判断语义各自有自己的稳定边界。

source_version

绑定事实本身的稳定版本。

不绑定 freshness、排名或随时间连续变化的分数。

set_id

绑定所有合法候选的 canonical membership。

同一批成员即使排名交叉，set_id 也不漂移。

review_id + review_version

只绑定本轮完整展示的 bounded review，以及 warning / overdue 这类离散判断语义。

隐藏候选没有被看见，不能被 quiet；旧 warning 不能签掉新 overdue。

09:59 Agent 看见 task X = warning

10:00 任务变 overdue

10:01 旧 review 必须 semantic_changed → 重新看

Attention 最怕的不是重复，而是静默丢失或假成功

所以它把“我看过”“我拿到了”“我做成了”拆成可审计状态。

Claim lease

打开 focus 后有租约。Agent 崩溃时，heartbeat 在下一轮 build 前恢复过期 claim，候选不会永久隐身。

Supersede freshness

同一 coalesce family 出现更新事实时，旧 claimed row 会变 stale，不能再写 canonical success。

Atomic settle

claim 校验、owner 更新与 receipt 在一个事务里完成；中途失败整笔 rollback。

Reviewed quiet

沉默也是终局，但只关闭 Agent 真正完整看过的 review membership。

available -> delivered -> selected -> requested -> executor_started
-> canonical_receipt -> visible_projection / deliberate_silence

代码 Green、已部署、已启用、自然行为已观察，是不同层级，不能互相冒充。

数据边界

一张 SQLite ，四个 owner

共享文件只是部署便利；AttentionCoordinator 不是所有表的主人。

InboxStore

外部事实 · 幂等 · 清洗 · coalesce · expiry

CalendarStore

明确提醒 · direct lane

ContinuationStore

有因果关系、明确延后的下一阶段

TaskStore

scheduled / standing / periodic + 独立 cycle

AttentionCoordinator

构建 deterministic view ，跨 owner 编排事务；exact claim / settle 仍回 source owner 。它不直接写 source rows 。

TaskStore.maintain() 在 AOS build 之前运行：维护动作不是“值得注意的事情”。

接信息、接能力、接渠道，各走自己的门

新项目不应该让 Attention core 横着长。

接外部信息源

external record → AgentEvent → InboxStore.ingest()

canonical commit 成功后才 ACK；adapter 失败不能挡住别的 owner。

接新能力

照常注册 Hermes MCP / tool / plugin。

Attention 只给 broad domain hint，不复制 schema、不授予权限、不维护白名单。

接新对话渠道

照常接入 Hermes；需要连续性时只重绑 native Cron origin。

不要新增 channel-specific owner，也不要吧群聊全文塞进 Inbox。

同一个系统可以换 QQ、mobile 或飞书；也可以接邮箱、论坛、报名网站而不重写“注意什么”这套核心。

接手代码，先顺着真实路径读

不要只凭 README 猜；下面这些文件就是生产路径。

`runtime.py`

heartbeat 顺序 / wake gate

`continuations.py`

deferred causal work

`attention.py`

Candidate / scoring / IDs / coordinator

`tasks.py`

scheduled / standing / periodic

`claims.py`

exact claim / lease / review semantics

`db.py`

schema / transactions / file modes

`inbox.py`

sanitization / idempotency / supersede

`adapters.py`

trusted registry / isolated polls

`calendar.py`

direct reminder owner

`cli.py`

arrange / build / focus surface

唯一数据库绑定

`install_hermes.py` 把 CLI 与 heartbeat 同时钉到一个 `HERMES_ATTENTION_DB`。自定义路径每次安装/升级都显式传 `--attention-db`；缺少绑定就 fail closed，不生成第二本账。

focus operations: open | validate | close | defer | quiet-set

最短冷启动

人类看懂全貌，AI 找到真代码

仓库新增 AI_START_HERE.md；它是路标，不是第二份产品合同。

1

AI_START_HERE.md

20 行以内，告诉新 Agent 先读什么、绝不能改坏什么。

2

architecture.md

canonical product contract：边界、owners、wake、tool exposure。

3

REQUIREMENTS + SCHEMA

把产品意图对到身份、事务、表结构和验收口径。

4

src + tests

沿真实入口验证；不要从旧日志或理论结构推断当前实现。

它最终想给 AI 的，不是更多指令，而是一种有证据的“此刻值得看一眼”。

github.com/Aryuan026/HermesAttentionCore